Princess Sumaya University for Technology
جامعة الأميرة سميّة للتكنولوجيا

11335 Operating Systems

Spring 2024-2025

Project Report

Performance Evaluation of CPU Scheduling Algorithms

Group Members:

Abdalrahman Alfahel	20220241	abd20220241@std.psut.edu.jo
Yousef Saddouq	20220187	you20220187@std.psut.edu.jo
Laila Sadideen	20220432	lai20220432@std.psut.edu.jo
Dana Salem	20220177	dan20220177@std.psut.edu.jo
Yazan Alfani	20220919	yaz20220919@std.psut.edu.jo

Table of Contents

Introduction

CPU scheduling is an essential part of how operating systems work. Computer systems continue to develop quickly, so we need more efficient management for scheduling the CPU processes. The main goal of CPU scheduling is to increase responsiveness, while decreasing wait and turnaround times.

There are two CPU scheduling approaches and they are used to manage the execution of the processes. Firstly, the non-preemptive approach which allows the process to complete its work without interruption. Secondly, the preemptive approach which switches between processes when needed.

Our report aims to evaluate the performance of different CPU scheduling algorithms and to calculate the average waiting and turnaround times. These calculations will be compared to understand how they work and how they differ from each other.

By analyzing results, we aim to determine which algorithms are more suitable for different types of systems. This analysis will help us in discovering the advantages and disadvantages of every scheduling algorithm, enabling us to choose the best method for maximizing CPU performance.

Methodology

1. First-Come First-Served (FCFS) – Non-Preemptive

Introduction

FCFS is the simplest CPU scheduling algorithm. In FCFS, the process that arrives first gets executed first. It's a **non-preemptive** scheduling algorithm, meaning once a process starts execution, it runs to completion. It operates much like a **FIFO** (First-In, First-Out) queue [1].

Pseudo Code

```
input n (number of processes)
for i = 0 to n-1:
    input arrival[i], burst[i]
    id[i] = i + 1
sort processes by arrival time
initialize:
    time = 0
    totalTurnaround = 0
    totalWaiting = 0
for i = 0 to n-1:
    if time < arrival[i]:
        time = arrival[i]
    completion[i] = time + burst[i]
    turnaround[i] = completion[i] - arrival[i]
    wait[i] = turnaround[i] - burst[i]
    time = completion[i]
```

```
totalTurnaround += turnaround[i]
totalWaiting += wait[i]
```

Print each process with: id, arrival, burst, completion, turnaround, wait
Compute and print average turnaround time and average waiting time

Advantages

1. **Simple to implement:** using a basic queue structure.
2. **Predictable behaviour:** due to strict arrival order.
3. **No context switching overhead:** since it's non-preemptive.

Disadvantages

1. **Poor performance:** if shorter processes wait for longer ones.
2. **Convoy Effect:** short processes wait behind long ones, increasing waiting time [1].
3. **Not suitable for time:** sharing systems due to non-preemptiveness.

2. Round Robin (RR) – Preemptive

Introduction

Round Robin (RR) is one CPU scheduler created mostly for preemptive systems sharing time. A fixed time quantum is assigned to each process within the ready queue, also called a time slice. That duration is processed by the CPU so, if unfinished, it gets preempted then moved to the queue’s end. All processes must be completed before this cycle stops.

Since Round Robin uses a time quantum, no process has a long wait for CPU time. It is because of this that it is suitable for multitasking environments. The quantum of time does affect performance and RR behaves in a manner like FCFS in the event it is too large. Frequent context switches as well as overhead result in the event it is too small.

Pseudo Code

```
input n (number of processes)
for i = 0 to n-1:
    input arrival[i]
for i = 0 to n-1:
    input burst[i]
    remaining[i] = burst[i]

input timeQuantum
initialize:
    time = 0
    finished = 0
    queue = empty
    inQueue[i] = false for all i
for i = 0 to n-1:
    if arrival[i] <= time and inQueue[i] == false:
        add i to queue
        inQueue[i] = true
while finished < n:
    if queue is empty:
        time += 1
        for i = 0 to n-1:
            if arrival[i] <= time and remaining[i] > 0 and inQueue[i] == false:
```

```
            add i to queue
            inQueue[i] = true
        continue
    idx = remove first element from queue
    if remaining[idx] > timeQuantum:
        time += timeQuantum
        remaining[idx] -= timeQuantum

    else:
        time += remaining[idx]
        remaining[idx] = 0
        completion[idx] = time
        turnaround[idx] = time - arrival[idx]
        wait[idx] = turnaround[idx] - burst[idx]
        finished += 1
    for i = 0 to n-1:
        if arrival[i] <= time and remaining[i] > 0 and inQueue[i] == false:
            add i to queue
            inQueue[i] = true
    if remaining[idx] > 0:
        add idx to queue
```

Print each process with: arrival, burst, wait, turnaround

Compute and print average waiting time and turnaround time

Advantages

1. **Fairness:** every process gets an equal share of the CPU so starvation is stopped.
2. **Good Response Time:** it is quite well-suited for the time-sharing systems because it ensures that each process runs inside of a bounded time frame.

3. **Predictable Behavior:** helpful in interactive systems where consistent response times improve user experience.

Disadvantages

1. **Performance Depends on Time Quantum:** choosing the optimal time quantum is challenging
    - If too large, it behaves like FCFS.
    - If too small, it causes excessive context switching
2. **Higher Turnaround Time:** RR often results in a higher average turnaround time as compared to algorithms such as Shortest Job First (SJF) especially in the case of long processes.
3. **Context Switching Overhead:** overhead increases because of frequent preemption that is due to time slices that are small, consuming CPU time in order to make each context switch.

3. Shortest-Job-First (SJF) – Non-Preemptive

Introduction

Shortest Job First is a CPU scheduling algorithm that chooses the process with the shortest burst time to be executed next in the central processing unit. When the CPU starts executing a non-preemptive version of Shortest Job First, it cannot be interrupted until the process is completed.

Pseudo Code

```
Input n (number of processes)
for i = 0 to n-1:
    Input arrival_time[i] and burst_time[i]
    id[i] = i + 1
Sort processes by arrival_time in ascending order
Initialize:
    current_time = 0
    completed_count = 0
    completed[i] = false for all i
while completed_count < n:
    id_index = -1
    minBurst = infinity
    for i = 0 to n-1:
        if completed[i] == false and arrival_time[i] <= current_time:
            if burst_time[i] < minBurst:
                minBurst = burst_time[i]
                id_index = i
    if id_index == -1:
        current_time += 1
        continue
    waiting_time[id_index] = current_time - arrival_time[id_index]
    current_time += burst_time[id_index]
    turnaround_time[id_index] = waiting_time[id_index] + burst_time[id_index]
    completed[id_index] = true
    completed_count += 1

Print each process with: arrival_time, burst_time, waiting_time, turnaround_time
Compute and print average waiting time and average turnaround time
```

Advantages

1. **Less average waiting time**: shorter processes are executed first, so the average waiting time is usually less than the other algorithms.
2. **Effective for short job processes**: it's efficient when most tasks have short burst times.
3. **Better average turnaround time:** since the non-preemptive version of Shortest Job First selects the shortest burst time, smaller processes complete faster leading to a reduction in the average turnaround time.

Disadvantages

1. **Starvation**: long processes might never be executed if there are many short jobs that continue to arrive.
2. **Hard to predict burst time**: in real-time systems, knowing the accurate burst time is difficult somehow.

4. Shortest-Remaining-Time-First (SRTF) – Preemptive

Introduction

The preemptive variant of the Shortest Job First (SJF) scheduling technique is called Shortest Remaining Time First (SRTF). The process with the smallest remaining burst duration is given the CPU in SRTF. The CPU is preempted and allocated to the new process if its burst time is less than the amount of time left in the running process. While a minimum average waiting time is guaranteed, longer processes

may suffer as a result. For systems where reducing response time is essential, it is perfect. Future burst times are necessary for SRTF, however this may not always be possible in practical situations.

Pseudo Code

```
input n (number of processes)
for i = 0 to n-1:
    input arrival[i] and burst[i]
    set remaining[i] = burst[i]
initialize:
    time = 0
    finished = 0
while finished < n:
    find process j with:
        - arrival[j] ≤ time
        - remaining[j] > 0
        - shortest remaining[j]
    if such process j exists:
        remaining[j] -= 1
        time += 1
        if remaining[j] == 0:
            turnaround[j] = time - arrival[j]
            wait[j] = turnaround[j] - burst[j]
            finished++
    else:
        time += 1
Print each process with: arrival, burst, wait, turnaround
Compute and print average waiting time and turnaround time
```

Advantages

1. **Minimizes Average Waiting Time:** SRTF reduces the average waiting time by prioritizing processes with the shortest remaining execution time.
2. **Efficient for Short Processes:** Shorter processes get completed faster, improving overall system responsiveness.
3. **Ideal for Time-Critical Systems:** It ensures that time-sensitive processes are executed quickly.[2]
4. Removes the **convoy** effect.

Disadvantages

1. **Difficult to Predict Burst Times:** Accurate prediction of process burst times is challenging and affects scheduling decisions.
2. **Starvation of Long Processes:** Longer processes may be delayed indefinitely if shorter processes keep arriving.
3. **Not Suitable for Real-Time Systems:** Real-time tasks may suffer delays due to frequent preemptions.[2]
4. **High Overhead:** Frequent context switching can increase overhead and slow down system performance.

5. Priority Scheduling – Non-Preemptive

Introduction

Each process is given a priority by the CPU scheduling technique known as Priority Scheduling (Non-Preemptive), which then allots the CPU to the process with the greatest priority (lowest numerical value, if lower number indicates higher priority). Once a process begins, it continues uninterrupted until it is finished. Setting priorities for important tasks is easy and efficient with this strategy. On the other hand, it might cause low-priority processes to starve.

Pseudo Code

```
input n (number of processes)
For i = 0 to n-1:
    input arrival[i], burst[i], priority[i]
    id[i] = i+1
    done[i] = 0
initialize t = 0, finished = 0
while finished < n:
    find process j with:
        - not done
        - arrival[j] <= t
        - lowest priority[j]
    if j!=-1:
        wait[j] = t - arrival[j]
        t += burst[j]
        turnAround[j] = t - arrival[j]
        done[j] = 1
```

```
        finished++
    else:
        t++
```

Print process details (id, arrival time, burst time, priority, waiting time, turnAround time)

Compute and print average waiting time and turnaround time

Advantages

1. **Quicker completion:** Priority scheduling ensures high priority processes are executed first. Therefore it has quicker completion of time-sensitive or critical tasks.
2. **Meet time constraints:** It is particularly useful for real-time systems that need to meet strict timing constraints.
3. **Speeds up overall execution:** Processes that require more CPU time can have their priority increased.

Disadvantages

1. **Delay may occur:** Priority inversion occurs when a lower priority process holds a resource needed by a higher priority process.
2. **Starvation:** If the system is overwhelmed with high priority processes lower priority processes may never get a chance to execute.
3. **Difficulty in setting priorities:** This is especially when dealing with many processes with varying priority levels.
4. **High complexity:** Techniques to avoid priority inversions, like priority inheritance or priority ceiling protocols, add complexity to the system.

Analysis and Results

This project explores, through the use of one unified dataset, the practical behavior of five classic CPU scheduling algorithms namely First Come First Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time First (SRTF), Round Robin (RR), and Priority Scheduling (Non-Preemptive). All algorithms were implemented using C++. This implementation guaranteed consistent structure and behavior across tests, plus it eliminated discrepancies that arose from language or environment differences.

The dataset consists of six processes with the following characteristics (All times used are in milliseconds):

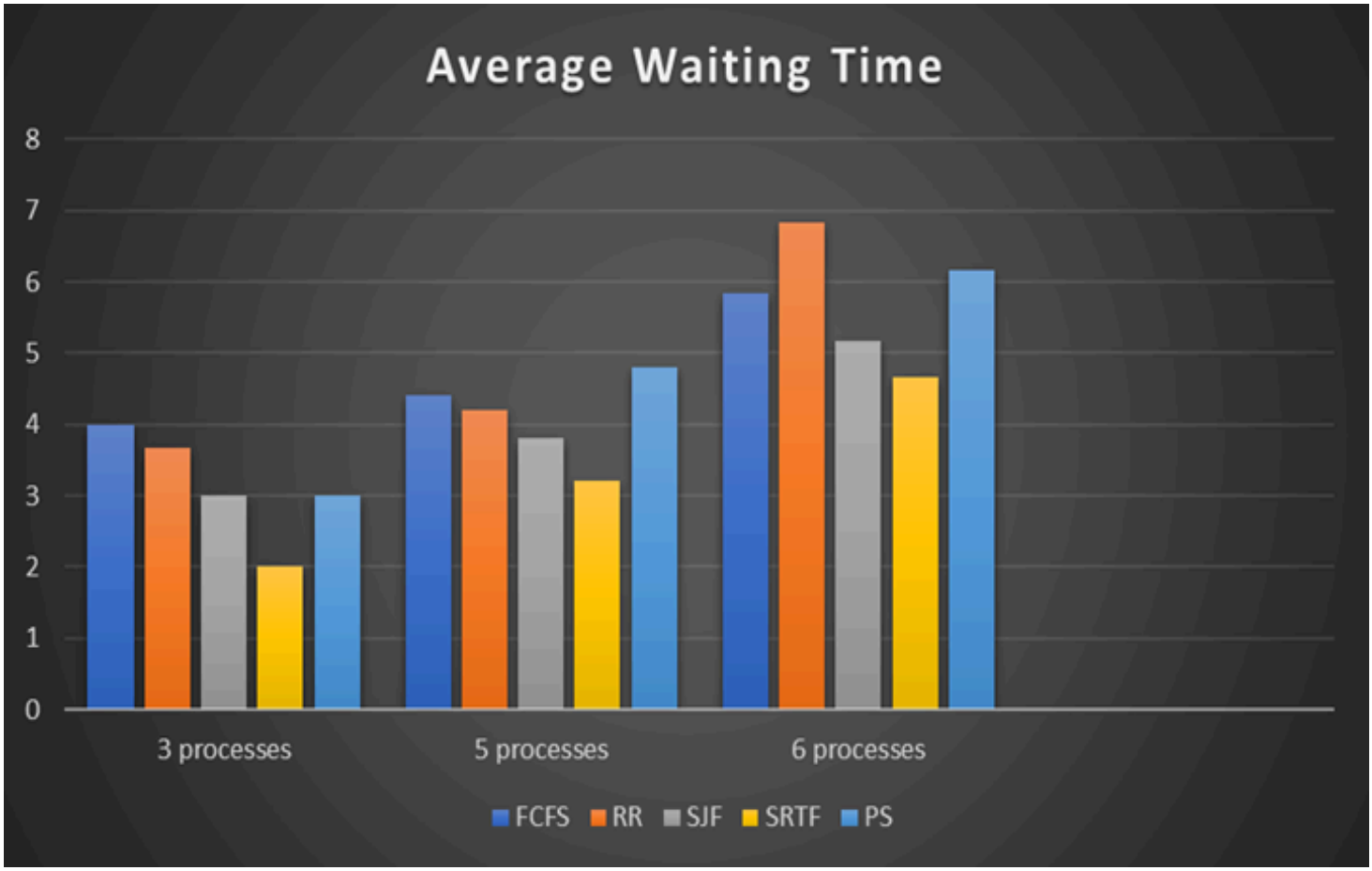
Process	Arrival Time	Burst Time	Priority (For Priority Scheduling – Non-Preemptive)
P1	0	7	3
P2	2	4	4
P3	4	1	1
P4	8	4	3
P5	10	9	2
P6	12	8	4

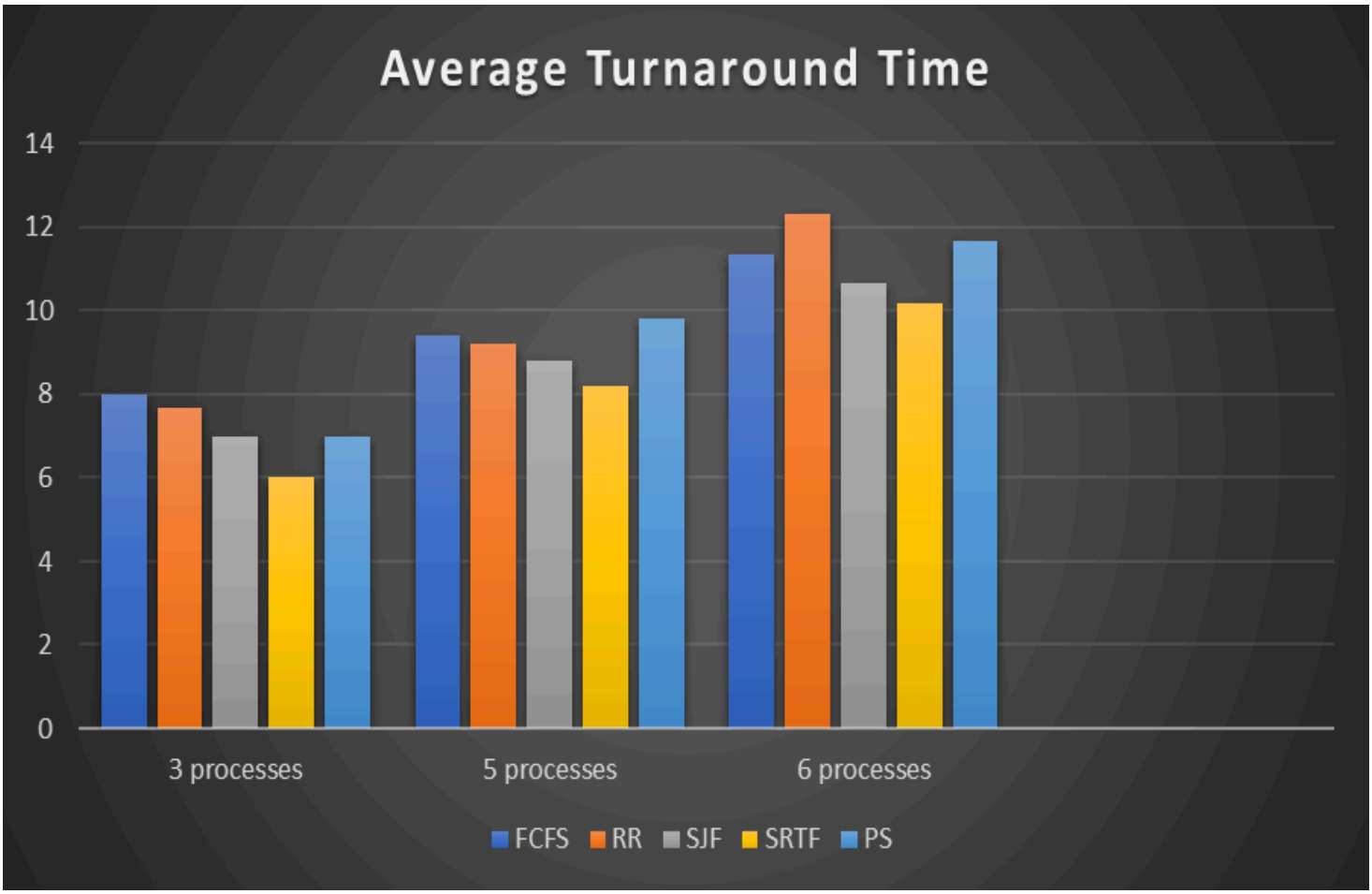
3 processes: P1,P2,P3.

5 processes: P1,P2,P3,P4,P5.

6 processes: P1,P2,P3,P4,P5,P6.

Time quantum = 4 (For RR).





This configuration includes a mix of short and long processes, varied arrival times, and differing priority levels—making it ideal for evaluating algorithm responsiveness, fairness, and efficiency.

Observations:

- SRTF excelled at minimizing individual job delays by choosing the job with the shortest remaining burst time always. For instance, P3 (burst = 1) arrived at time 4, also it quickly preempted longer jobs like P2 or P1, finishing almost immediately. However, jobs such as P5 (burst = 9) faced frequent delays, and that exposed the risk of starvation.

- SJF prioritized short jobs with effectiveness when they arrived early. It remained non-preemptive, however. P3 was the choice that was made over other options. Its burst time provided a vital edge. However, once a long process was started by someone, like P5, others had to wait for it, so responsiveness was reduced.

- FCFS executed processes in strict order of arrival. This meant P1 was executed fully before any other process, even though shorter jobs like P3 arrived soon after. This clearly demonstrated the convoy effect, where longer jobs block shorter, more efficient ones from running sooner.

- Round Robin (RR) used a fixed time quantum of 4. Processes like P2 (burst = 4) completed in one cycle, while longer processes like P5 and P6 were split into multiple cycles. The algorithm prevented starvation and then overhead increased since the algorithm did frequently switch contexts. Requeuing jobs like P5 occurred with prominent frequency. These jobs failed for completion.

- Processes did run based upon static priority values with Priority Scheduling (Non-Preemptive). P3 (priority = 1) was executed fast even if it arrived later than P2 however. Meanwhile, lower-priority jobs like P2 and P6 waited until all higher-priority jobs completed, showing how this algorithm can improve response for important tasks but also starve lower-priority ones.

This accurate analysis validates each algorithm's distinct response when facing identical demands. Longer tasks risk starvation though SRTF as well as SJF handle short ones efficiently. Though fairness is ensured, efficiency suffers in FCFS. RR finds equilibrium however prefers a quantum greatly. Priority Scheduling favors urgency. However, careful management prevents unfair delays.

Original Contribution

- **Unified Implementation:** All algorithms were coded the same input processes to achieve a fair comparison. This eliminated variability in results that could happen from different datasets.
- **Performance Comparison:** A detailed comparison study was made between the five algorithms using real input values. This study revealed that SRTF is the most time-efficient but also showed its complexity and unfairness, and analysed the performance of Round Robin with different quantum values, with an emphasis on how system performance is affected by parameter tuning.
- **Bridging Theory and Practice**: While most academic resources focus on theoretical advantages, this report focuses on how these algorithms behave when implemented and evaluated with actual data, showing the difference between theoretical and real-world performance.

This original work is a valuable resource not only for understanding how each scheduling algorithm works, but also how it performs under practical conditions, offering readers both technical and decision-making knowledge.

Conclusion

We thoroughly examined five core CPU scheduling algorithms within this project: FCFS, RR, SJF, SRTF, and even Priority Scheduling (Non-Preemptive). By putting each algorithm into practice then evaluating it on a consistent dataset of processes, we were able to accurately compare them in terms of average waiting time and average return time. Our findings provide support. Theoretical predictions are supported. SRTF, as an instance, continuously offered up the lowest average turnaround and waiting times because of its dynamic handling of shorter processes however introduced meaningful complexity and the risk of starving for activities that are longer.

Round robin was very sensitive to the chosen time slice and had more overhead, though fair and ideal for time-sharing systems. FCFS was straightforward. However, it did battle within mixed workloads because of the convoy effect. SJF performed exceptionally well for brief tasks, though it struggled in adjusting to unpredictable burst times. Priority Scheduling (Non-Preemptive) stressed how important—and difficult—it is to allocate priorities accurately since failing to do so could cause lower-priority jobs to go hungry, last but not least.

In a practical way of speaking, not a single algorithm outperformed all of the others. Responsiveness or throughput, or even fairness, are some objectives of a system within its particular context. The optimal scheduling approach is largely influenced by these factors. This effort did improve on our comprehension of CPU scheduling trade-offs as well as the difficulties that come from putting ideal algorithms into practice.

References

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.

[2] Coding Ninjas. (n.d.). *Non-Preemptive Priority Based Scheduling*. Naukri Code 360.

URL addresses:

- https://www.geeksforgeeks.org/round-robin-scheduling-in-operating-system/
- https://www.naukri.com/code360/library/non-preemptive-priority-based-scheduling